

Методичка 15. Telegram-боты на Python

1. Зачем нужна эта тема

Telegram-боты позволяют создавать программы, с которыми пользователь общается через Telegram.

Бот может:

- отвечать на команды;
- показывать меню;
- принимать текстовые сообщения;
- задавать вопросы;
- сохранять ответы пользователя;
- отправлять кнопки;
- обрабатывать нажатия на кнопки;
- проводить регистрацию;
- делать викторины;
- показывать расписание;
- принимать заявки.

Telegram-бот — хороший учебный формат, потому что ученик видит результат не только в консоли, а в настоящем приложении.

Эта тема связана с предыдущими темами:

- переменные нужны для хранения данных;
- условия — для обработки команд;
- функции — для разделения логики;
- словари и списки — для хранения пользователей и заявок;
- ООП может использоваться для более крупной структуры;
- исключения нужны, чтобы бот не падал при ошибках.

2. Главная идея темы

Telegram-бот работает по принципу событий.

Пользователь что-то делает:

- отправляет команду;
- пишет сообщение;
- нажимает кнопку.

Бот получает это событие и запускает нужную функцию-обработчик.

Бот — программа, которая общается с пользователем в Telegram.

Токен — специальный ключ, по которому Telegram понимает, каким ботом управляет программа.

Сообщение — текст или команда от пользователя.

Хендлер / обработчик — функция, которая реагирует на сообщение или кнопку.

Команда — сообщение, которое начинается с `/`, например `/start`.

Reply-кнопка — кнопка, которая отправляет обычный текст от имени пользователя.

Inline-кнопка — кнопка под сообщением, которая отправляет callback-данные.

Callback — событие, которое возникает при нажатии inline-кнопки.

Состояние пользователя — этап сценария, на котором сейчас находится пользователь.

3. Базовый синтаксис

Установка библиотеки

Обычно для работы с Telegram-ботами используется библиотека `pyTelegramBotAPI`.

Установка:

```
pip install pyTelegramBotAPI
```

Импорт и создание бота

```
import telebot

TOKEN = "сюда_вставляется_токен"

bot = telebot.TeleBot(TOKEN)
```

Что происходит:

- импортируется библиотека;
- токен сохраняется в переменную;
- создаётся объект бота.

Важно: токен нельзя публиковать в открытом доступе.

Обработка команды `/start`

```
@bot.message_handler(commands=[ "start" ])
def start(message):
    bot.send_message(message.chat.id, "Привет!")
```

Что происходит:

- декоратор говорит: эта функция реагирует на команду `/start` ;
- `message` содержит информацию о сообщении;
- `message.chat.id` нужен, чтобы бот понял, куда отправить ответ;
- `send_message()` отправляет сообщение пользователю.

Запуск бота

```
bot.infinity_polling()
```

Эта команда запускает постоянную работу бота.

Бот начинает ждать сообщения.

4. Разбор темы от простого к сложному

Шаг 1. Самый простой бот

```
import telebot

TOKEN = "сюда_вставляется_токен"

bot = telebot.TeleBot(TOKEN)

@bot.message_handler(commands=["start"])
def start(message):
    bot.send_message(message.chat.id, "Привет! Я бот.")

bot.infinity_polling()
```

Бот отвечает на команду `/start` .

Шаг 2. Ответ на текст

```
@bot.message_handler(content_types=["text"])
def handle_text(message):
    bot.send_message(message.chat.id, "Вы написали: " + message.text)
```

Этот обработчик реагирует на обычный текст.

`message.text` — текст, который отправил пользователь.

Шаг 3. Условия внутри обработчика

```
@bot.message_handler(content_types=["text"])
def handle_text(message):
    if message.text == "Привет":
        bot.send_message(message.chat.id, "Привет!")
    elif message.text == "Помощь":
        bot.send_message(message.chat.id, "Список команд: /start")
    else:
        bot.send_message(message.chat.id, "Неизвестная команда")
```

Бот может выбирать ответ в зависимости от текста.

Шаг 4. Reply-кнопки

```
from telebot import types

@bot.message_handler(commands=["menu"])
def menu(message):
    markup = types.ReplyKeyboardMarkup(resize_keyboard=True)

    button1 = types.KeyboardButton("Привет")
    button2 = types.KeyboardButton("Помощь")

    markup.add(button1, button2)

    bot.send_message(message.chat.id, "Выберите действие:", reply_markup=markup)
```

Reply-кнопки появляются вместо клавиатуры. Когда пользователь нажимает кнопку, в чат отправляется обычный текст.

Шаг 5. Inline-кнопки

```
from telebot import types

@bot.message_handler(commands=["inline"])
def inline_menu(message):
    markup = types.InlineKeyboardMarkup()

    button1 = types.InlineKeyboardButton("Да", callback_data="yes")
    button2 = types.InlineKeyboardButton("Нет", callback_data="no")

    markup.add(button1, button2)

    bot.send_message(message.chat.id, "Выберите вариант:", reply_markup=markup)
```

Inline-кнопки находятся под сообщением.

Они не отправляют обычный текст в чат. Вместо этого они отправляют callback-данные.

Шаг 6. Обработка callback

```
@bot.callback_query_handler(func=lambda call: True)
def handle_callback(call):
    if call.data == "yes":
        bot.send_message(call.message.chat.id, "Вы выбрали да")
    elif call.data == "no":
        bot.send_message(call.message.chat.id, "Вы выбрали нет")
```

`call.data` хранит значение `callback_data`, которое было указано в кнопке.

Шаг 7. Хранение данных пользователя

```
users = {}

@bot.message_handler(commands=["start"])
def start(message):
    user_id = message.from_user.id

    users[user_id] = {
        "name": None,
        "age": None
    }

    bot.send_message(message.chat.id, "Регистрация началась")
```

Словарь может хранить данные пользователей.

Ключом удобно использовать `user_id`.

Шаг 8. Простые состояния

```

users = {}

@bot.message_handler(commands=["register"])
def register(message):
    user_id = message.from_user.id

    users[user_id] = {
        "state": "wait_name",
        "name": None,
        "age": None
    }

    bot.send_message(message.chat.id, "Введите имя")

@bot.message_handler(content_types=["text"])
def handle_text(message):
    user_id = message.from_user.id

    if user_id not in users:
        bot.send_message(message.chat.id, "Введите /register")
        return

    state = users[user_id]["state"]

    if state == "wait_name":
        users[user_id]["name"] = message.text
        users[user_id]["state"] = "wait_age"
        bot.send_message(message.chat.id, "Введите возраст")

    elif state == "wait_age":
        users[user_id]["age"] = message.text
        users[user_id]["state"] = "done"
        bot.send_message(message.chat.id, "Регистрация завершена")

```

Состояние помогает понять, какой ответ сейчас ждёт бот.

Шаг 9. Админский просмотр данных

```
ADMIN_ID = 123456789
```

```
@bot.message_handler(commands=[ "users" ])
def show_users(message):
    if message.from_user.id != ADMIN_ID:
        bot.send_message(message.chat.id, "Нет доступа")
        return

    for user_id, data in users.items():
        text = f"{user_id}: {data}"
        bot.send_message(message.chat.id, text)
```

Так можно сделать простую команду для просмотра данных.

5. Частые ошибки учеников

Ошибка 1. Публикуют токен

Неправильно:

```
TOKEN = "123456:ABC_SECRET_TOKEN"
```

Почему это опасно:

Если токен попадёт в открытый доступ, чужой человек сможет управлять ботом.

Правильно:

```
TOKEN = "сюда_вставляется_токен"
```

В реальных проектах токен лучше хранить отдельно от кода.

Ошибка 2. Забывают запустить polling

Неправильно:

```
@bot.message_handler(commands=[ "start" ])
def start(message):
    bot.send_message(message.chat.id, "Привет")
```

Почему это ошибка:

Обработчик есть, но бот не запущен.

Правильно:

```
bot.infinity_polling()
```

Ошибка 3. Забывают `message.chat.id`

Неправильно:

```
bot.send_message("Привет")
```

Почему это ошибка:

Бот должен знать, в какой чат отправить сообщение.

Правильно:

```
bot.send_message(message.chat.id, "Привет")
```

Ошибка 4. Путают `reply`-кнопки и `inline`-кнопки

Неправильно ожидать, что `reply`-кнопка даст `callback_data`.

```
button = types.KeyboardButton("Да")
```

Почему это важно:

`KeyboardButton` отправляет текст в чат. `InlineKeyboardButton` отправляет `callback`-данные.

Правильно для `callback`:

```
button = types.InlineKeyboardButton("Да", callback_data="yes")
```

Ошибка 5. Создали `inline`-кнопку, но не сделали `callback`-обработчик

Неправильно:

```
button = types.InlineKeyboardButton("Да", callback_data="yes")
```

Почему это ошибка:

Кнопка есть, но бот не знает, что делать после нажатия.

Правильно:

```
@bot.callback_query_handler(func=lambda call: True)
def handle_callback(call):
    if call.data == "yes":
        bot.send_message(call.message.chat.id, "Вы выбрали да")
```

Ошибка 6. Не проверяют состояние пользователя

Неправильно:

```
@bot.message_handler(content_types=["text"])
def handle_text(message):
    users[message.from_user.id]["name"] = message.text
```

Почему это ошибка:

Если пользователь ещё не начал регистрацию, его может не быть в словаре.

Правильно:

```
@bot.message_handler(content_types=["text"])
def handle_text(message):
    user_id = message.from_user.id

    if user_id not in users:
        bot.send_message(message.chat.id, "Сначала введите /register")
        return
```

Ошибка 7. Один обработчик перехватывает всё слишком рано

Неправильно:

```
@bot.message_handler(content_types=["text"])
def handle_all(message):
    bot.send_message(message.chat.id, "Любой текст")
```

Почему это может мешать:

Если обработчики расположены неаккуратно, общий обработчик может мешать более конкретной логике.

Лучше явно проверять команды, состояния и текст.

Ошибка 8. Не обрабатывают неправильный ввод

Неправильно:

```
users[user_id]["age"] = int(message.text)
```

Почему это ошибка:

Если пользователь введёт текст, бот может упасть.

Правильно:

```
try:
    age = int(message.text)
    users[user_id]["age"] = age
except ValueError:
    bot.send_message(message.chat.id, "Возраст нужно ввести числом")
```

6. Мини-примеры для закрепления

Пример 1. Ответ на `/start`

```
@bot.message_handler(commands=["start"])
def start(message):
    bot.send_message(message.chat.id, "Привет!")
```

Закрепляется обработка команды.

Пример 2. Эхо-бот

```
@bot.message_handler(content_types=["text"])
def echo(message):
    bot.send_message(message.chat.id, message.text)
```

Бот повторяет текст пользователя. Закрепляется `message.text`.

Пример 3. Простое меню через текст

```
@bot.message_handler(content_types=["text"])
def handle_text(message):
    if message.text == "1":
        bot.send_message(message.chat.id, "Раздел 1")
    elif message.text == "2":
        bot.send_message(message.chat.id, "Раздел 2")
    else:
        bot.send_message(message.chat.id, "Неизвестная команда")
```

Закрепляются условия внутри обработчика.

Пример 4. Reply-кнопки

```
markup = types.ReplyKeyboardMarkup(resize_keyboard=True)
markup.add(types.KeyboardButton("Помощь"))

bot.send_message(message.chat.id, "Меню:", reply_markup=markup)
```

Закрепляется создание обычной клавиатуры.

Пример 5. Inline-кнопка

```
markup = types.InlineKeyboardMarkup()
markup.add(types.InlineKeyboardButton("Нажми", callback_data="click"))

bot.send_message(message.chat.id, "Кнопка:", reply_markup=markup)
```

Закрепляется inline-кнопка.

Пример 6. Callback

```
@bot.callback_query_handler(func=lambda call: True)
def callback(call):
    if call.data == "click":
        bot.send_message(call.message.chat.id, "Кнопка нажата")
```

Закрепляется обработка нажатия inline-кнопки.

Пример 7. Словарь пользователей

```
users = {}

@bot.message_handler(commands=["start"])
def start(message):
    user_id = message.from_user.id
    users[user_id] = {}

    bot.send_message(message.chat.id, "Пользователь добавлен")
```

Закрепляется хранение данных по `user_id`.

7. Практические задания

Лёгкий уровень

1. Создать бота, который отвечает на команду `/start`.
2. Создать бота, который повторяет любой текст пользователя.
3. Добавить команду `/help`, которая выводит список команд.
4. Сделать текстовое меню: пользователь вводит `1`, `2` или `0`.
5. Добавить reply-кнопку `"Помощь"`.

Средний уровень

1. Сделать inline-кнопки `"Да"` и `"Нет"` и обработать нажатия.
2. Сделать мини-анкету: бот спрашивает имя и возраст.
3. Сохранять данные пользователя в словарь.
4. Проверять, что возраст введён числом.
5. Сделать команду `/profile`, которая показывает сохранённые данные пользователя.

Повышенный уровень

1. Сделать регистрацию на событие: имя, возраст, направление.
2. Добавить inline-кнопки для выбора направления.
3. Сделать команду администратора для просмотра всех заявок.
4. Добавить простую пагинацию: показывать заявки по одной.
5. Сделать защиту: админ-команды доступны только одному ID.

8. Контрольные вопросы

1. Что такое Telegram-бот?

2. Что такое токен?
3. Почему токен нельзя публиковать?
4. Что такое обработчик сообщения?
5. Что делает `@bot.message_handler` ?
6. Для чего нужен `message.chat.id` ?
7. Что хранится в `message.text` ?
8. Чем reply-кнопка отличается от inline-кнопки?
9. Что такое `callback_data` ?
10. Что делает `@bot.callback_query_handler` ?
11. Для чего нужны состояния пользователя?
12. Почему данные пользователей удобно хранить по `user_id` ?

9. Краткий конспект в конце

Главная идея темы: Telegram-бот реагирует на события пользователя: команды, текст и нажатия кнопок.

Минимальная структура:

```
import telebot

TOKEN = "сюда_вставляется_токен"

bot = telebot.TeleBot(TOKEN)

@bot.message_handler(commands=["start"])
def start(message):
    bot.send_message(message.chat.id, "Привет!")

bot.infinity_polling()
```

Reply-кнопки:

```
markup = types.ReplyKeyboardMarkup(resize_keyboard=True)
markup.add(types.KeyboardButton("Помощь"))
```

Inline-кнопки:

```
markup = types.InlineKeyboardMarkup()
markup.add(types.InlineKeyboardButton("Да", callback_data="yes"))
```

Callback:

```
@bot.callback_query_handler(func=lambda call: True)
def callback(call):
    if call.data == "yes":
        bot.send_message(call.message.chat.id, "Да")
```

Важно запомнить:

- бот работает через обработчики;
- команда начинается с `/`;
- `message.chat.id` нужен для ответа;
- `message.text` хранит текст пользователя;
- reply-кнопки отправляют текст;
- inline-кнопки отправляют callback;
- состояние пользователя помогает вести сценарий;
- токен нельзя публиковать.

Частые ошибки:

- забыть `bot.infinity_polling()`;
- забыть `message.chat.id`;
- перепутать reply и inline-кнопки;
- не обработать callback;
- не проверить состояние пользователя;
- не обработать неправильный ввод;
- случайно опубликовать токен.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- создавать простого Telegram-бота;
- подключать `telebot`;
- обрабатывать команду `/start`;
- отвечать на текстовые сообщения;
- создавать reply-кнопки;
- создавать inline-кнопки;
- обрабатывать callback-нажатия;
- хранить данные пользователя в словаре;
- делать простой сценарий регистрации;
- проверять ввод пользователя;
- понимать базовую структуру Telegram-бота.

